Object Oriented Programming

ICT2122

Introduction to Java Database Connectivity

P.H.P. Nuwan Laksiri
Department of ICT
Faculty of Technology
University of Ruhuna

Lesson 07

Recap – File Handling

- Getting to Know About Streams
- *Streams*
  - *Character Streams*
  - *Binary Streams*
- *Character Streams*
  - *Reading*
  - *Writing*
- *Binary Streams*
  - *Reading*
  - *Writing*

Outline

- What is an API
- JDBC Introduction
- Why JDBC
- JDBC Components
- JDBC Architecture
- JDBC Drivers
- Basic steps to use a database in Java
- Handling Exceptions

API – What is it?

- API stands for application programming interface.
- Java SE API
  - The Java Platform, Standard Edition (Java SE) APIs define the core Java platform for general-purpose computing.
  - These APIs are in modules whose names start with ***java***.

  https://docs.oracle.com/en/java/javase/27/docs/api/index.html

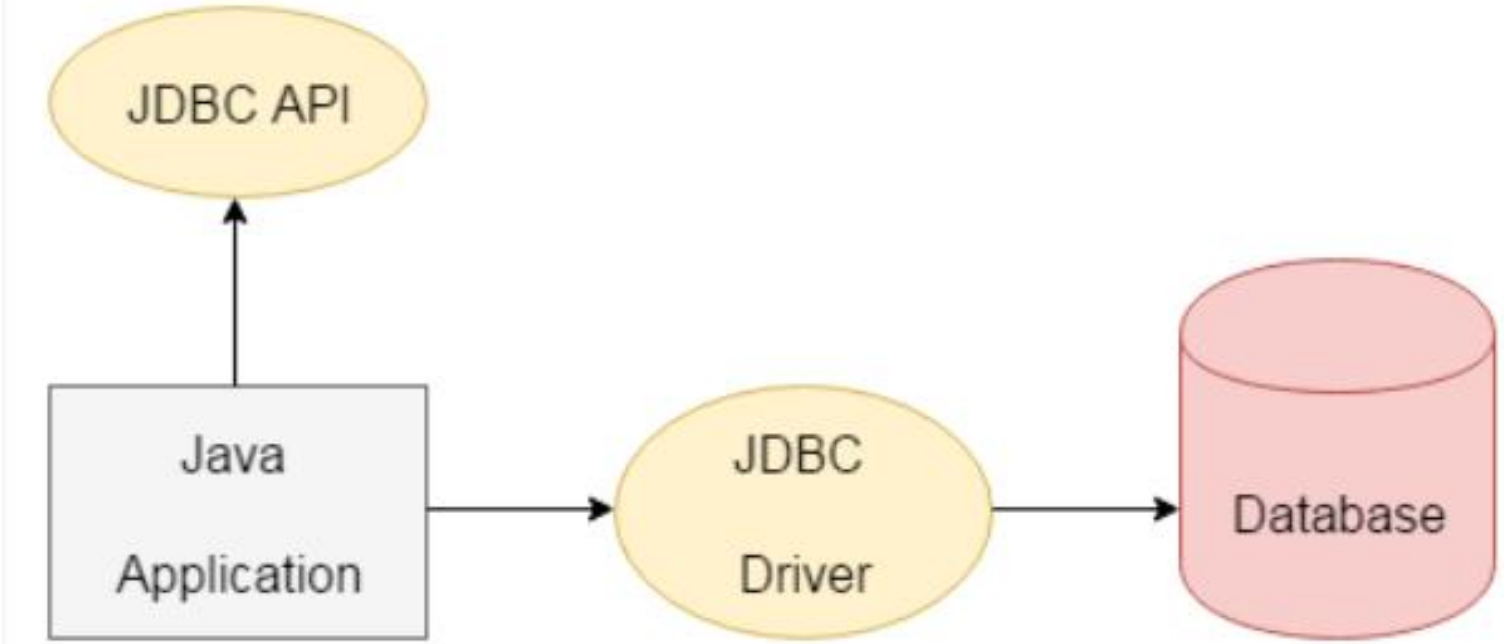
- On the other hand, API is a document that contains description of all the features of a product or software.
- It represents classes and interfaces that software programs can follow to communicate with each other.
- An API can be created for applications, libraries, operating systems, etc.

API – What is it?

- The Java language itself is very simple,
  - but Java comes with a library of classes that provide commonly used utility functions that most Java programs can't do without.
- This class library, called the Java API is as much a part of Java as the language itself.
- The real challenge of finding out how to use Java isn't mastering the language; it's mastering the API.
- The Java language has only about 50 keywords, but the Java API has several thousand classes, with tens of thousands of methods that you can use in your programs.

JDBC Introduction

- JDBC stands for *Java Database Connectivity*, which is a standard Java API for database-independent connectivity between the Java programming language and a wide range of databases.
- JDBC is to connect and execute query with the database.
- JDBC API uses jdbc drivers to connect with the database.



JDBC Introduction

- JDBC allows to,
  - Establishing a connection with a database or other tabular data source
  - Sending SQL commands to the database
  - Processing the results

Why JDBC

- Before JDBC, ODBC(*Open Database Connectivity)* API was the database API to connect and execute query with the database.

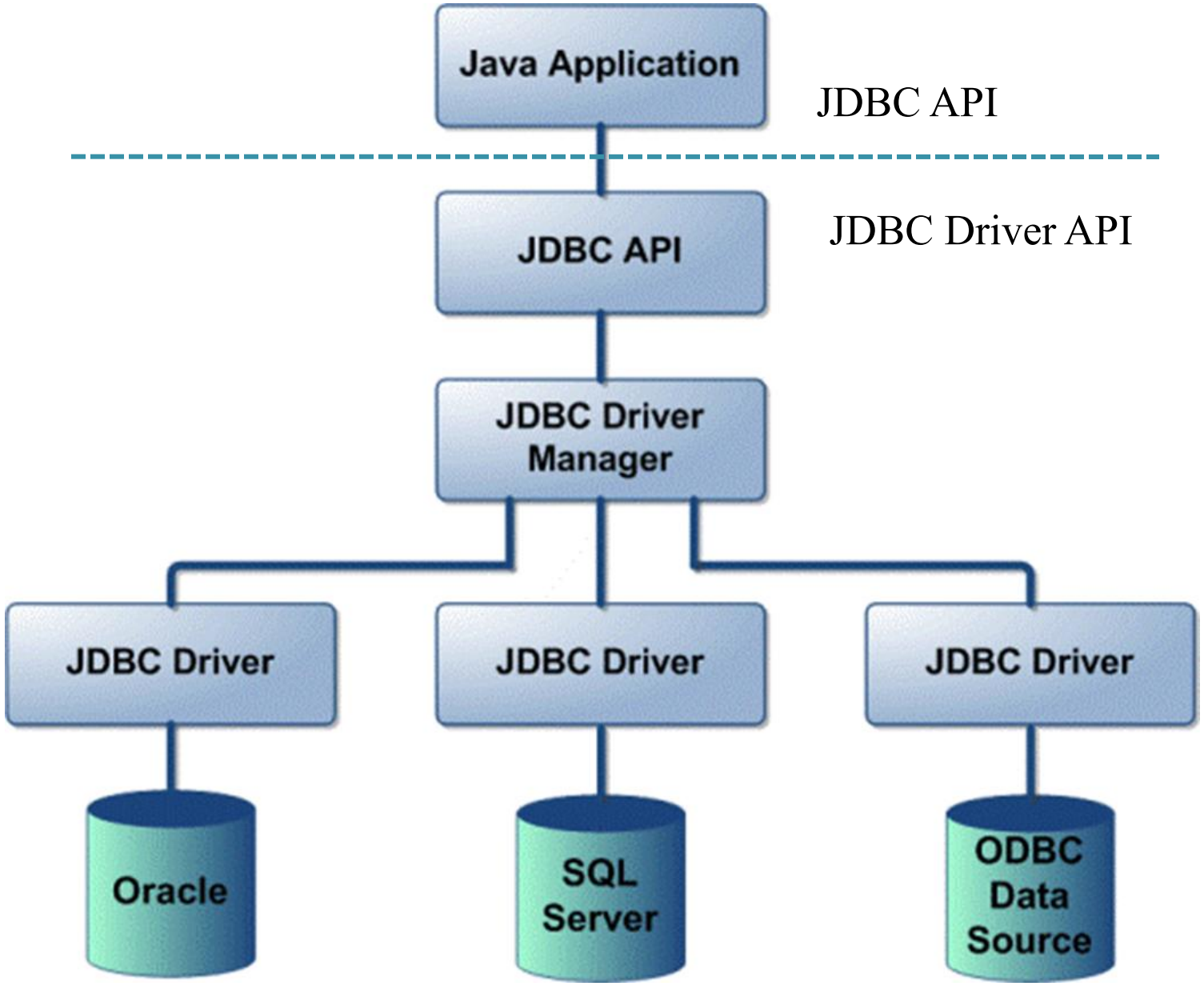
- But ODBC API uses ODBC driver which is written in C language (it's platform dependent and unsecured).

- That is why Java has defined its own API (JDBC API) that uses JDBC drivers (written in Java language).

JDBC Architecture Components

- JDBC API.
  - A set of Java classes and interfaces used to connect and execute queries with databases.
- JDBC DriverManager.
  - Manages database drivers and establishes connections between Java applications and databases.
- JDBC Driver
  - A software component that enables Java applications to communicate with a specific database.
- JDBC Test Suite.
  - A tool used to test whether JDBC drivers correctly implement the API.
- JDBC-ODBC Bridge.
  - An old method used to connect to databases via ODBC. Not used in modern Java.

JDBC Architecture



JDBC Architecture

- JDBC architecture consists of two layers,
  - JDBC API:

    Provides the connection between the Java application and the DriverManager.

    JDBC Driver API:

    Provides the connection between the DriverManager and the database drivers.

- The JDBC API uses
  - DriverManager
  - Database-specific drivers

  This allows Java applications to connect to different types of databases easily..

- Role of DriverManager
  - Selects the correct driver for the database
  - Establishes the connection
  - Supports multiple drivers at the same time
  - Enables connection to different databases

Common JDBC components

- The JDBC API provides the following interfaces and classes,

- **DriverManager**
  - This class manages a list of database drivers.
  - Matches connection requests from the java application with the proper database driver using communication sub protocol.
  - The first driver that recognizes a certain sub protocol under JDBC will be used to establish a database Connection.

- **Driver**
  - This interface handles the communications with the database server.

Common JDBC components

- **Connection**
  - This interface with all methods for contacting a database.
  - The connection object represents communication context, i.e., all communication with database is through connection object only.

- **Statement**
  - User use objects created from this interface to submit the SQL statements to the database.
  - Some derived interfaces accept parameters in addition to executing stored procedures.

Common JDBC components

- **ResultSet**
  - These objects hold data retrieved from a database after you execute an SQL query using Statement objects.

- **SQLException**
  - This class handles any errors that occur in a database application.

JDBC Driver

- JDBC Driver is a software component that enables java application to interact with a database.
- There are 4 types of JDBC drivers.
  - JDBC-ODBC Bridge Driver
  - Native-API Driver

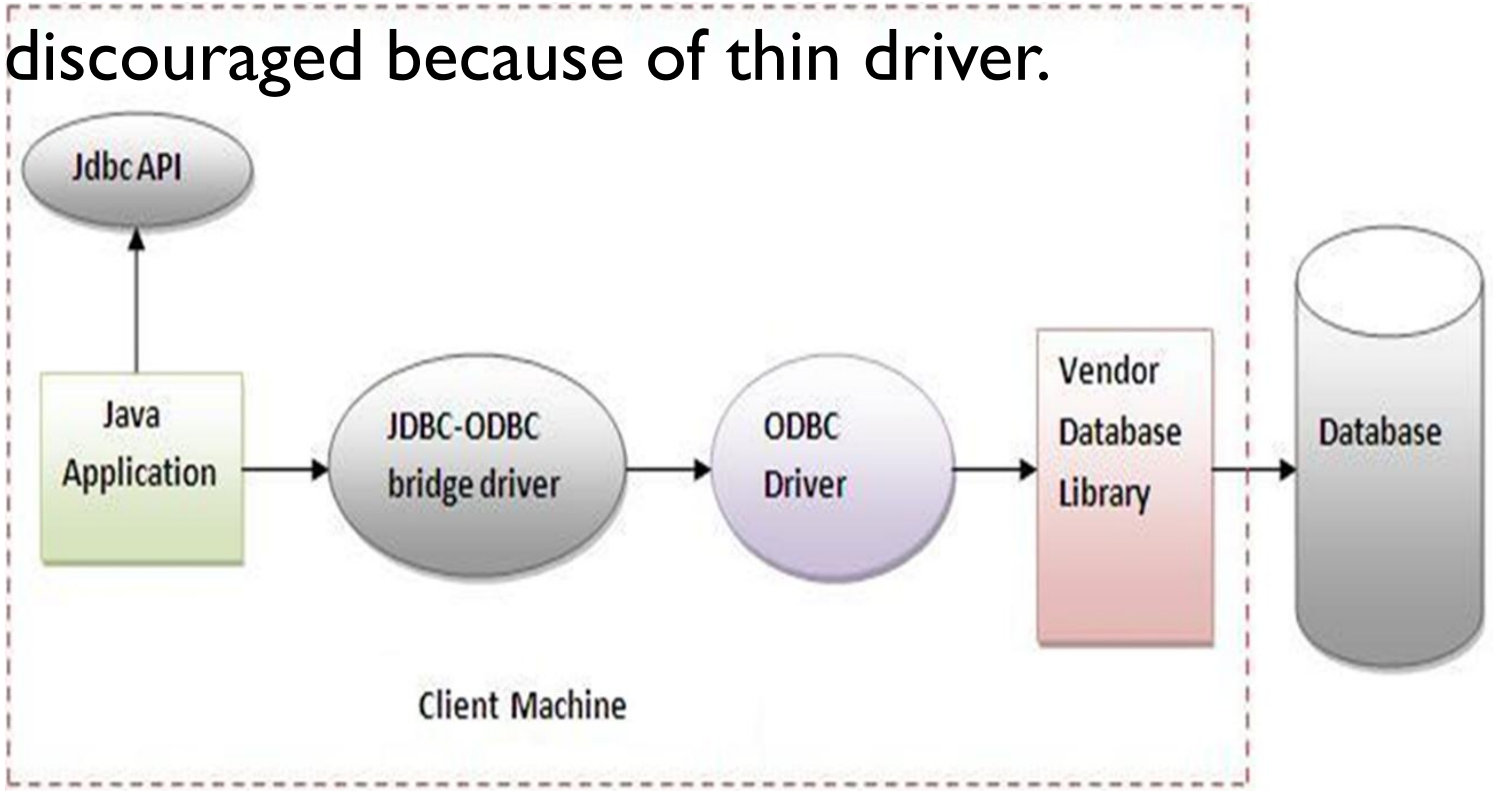
    (partially java driver)
  - Network Protocol Driver

    (fully java driver)
  - Thin Driver

    (fully java driver)

JDBC Driver - JDBC-ODBC bridge driver

- The JDBC-ODBC bridge driver uses ODBC driver to connect to the database.
- The JDBC-ODBC bridge driver converts JDBC method calls into the ODBC function calls.
- This is now discouraged because of thin driver.



JDBC Driver - JDBC-ODBC bridge driver

Advantages

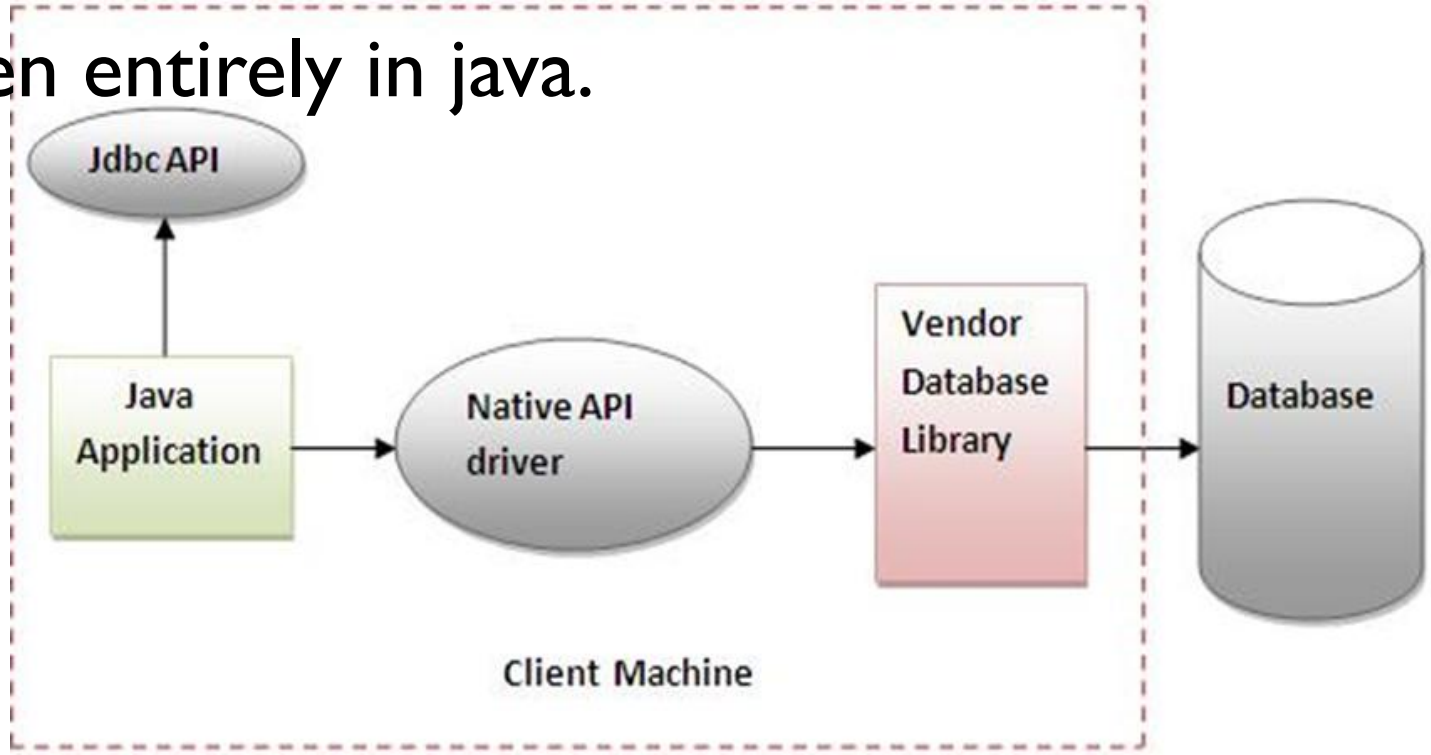
- Easy to use.
- Can be easily connected to any database.

Disadvantages

- Performance degraded because JDBC method call is converted into the ODBC function calls.
- The ODBC driver needs to be installed on the client machine.

JDBC Driver – Native API driver

- The Native API driver uses the client-side libraries of the database.
- The driver converts JDBC method calls into native calls of the database API.
- It is not written entirely in java.



JDBC Driver – Native API driver

Advantages

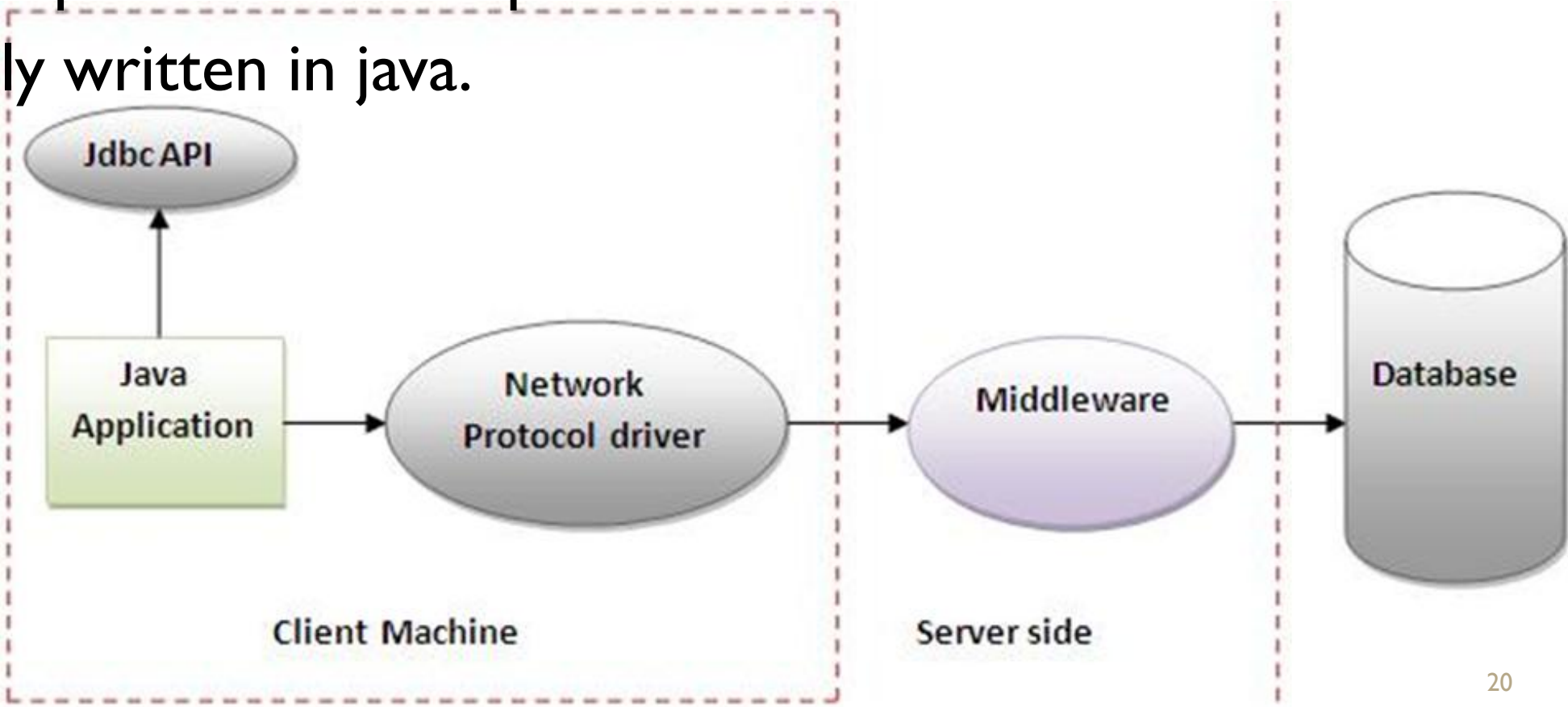
- Performance upgraded than JDBC-ODBC bridge driver.

Disadvantages

- The Native driver needs to be installed on each client machine.
- The Vendor client library needs to be installed on client machine.

JDBC Driver – Network Protocol driver

- The Network Protocol driver uses middleware (application server) that converts JDBC calls directly or indirectly into the vendor-specific database protocol.
- It is fully written in java.



JDBC Driver – Network Protocol driver

Advantages

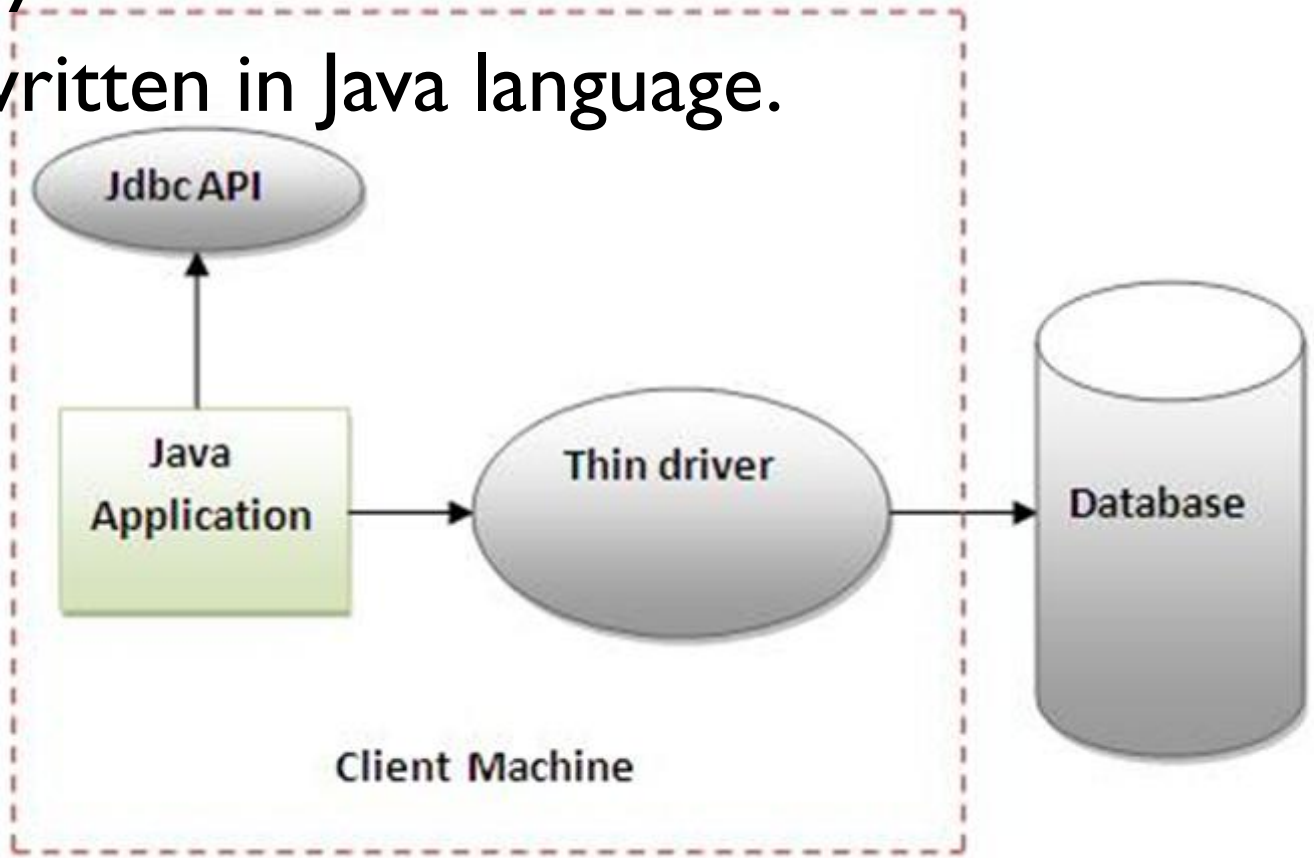
- No client-side library is required because of application server that can perform many tasks like auditing, load balancing, logging etc.

Disadvantages

- Network support is required on client machine.
- Requires database-specific coding to be done in the middle tier.
- Maintenance of Network Protocol driver becomes costly because it requires database-specific coding to be done in the middle tier.

JDBC Driver – Thin driver

- The thin driver converts JDBC calls directly into the vendor-specific database protocol.
- That is why it is known as thin driver.
- It is fully written in Java language.



JDBC Driver – Thin driver

Advantages

- Better performance than all other drivers.
- No software is required at client side or server side.

Disadvantages

- Drivers depend on the Database.

MySQL Connector/J

- MySQL Connector/J provides connectivity for Java applications to MySQL databases.

- It implements the Java Database Connectivity (JDBC) API along with some useful extensions.
- Connector/J is a JDBC Type 4 driver, which means:
  - It is a pure Java implementation of the MySQL protocol.
  - It does not require any native MySQL client libraries.

Basic Steps to Use a Database in Java

- There are 05 basic steps to connect a Java application to a database using JDBC:

1. **Register the Driver Class**
2. **Create a Connection**
3. **Create a Statement**
4. **Execute Queries**
5. **Close the Connection**

Register the Driver Class

- In JDBC, you need to register the driver, so Java knows which database to connect to.
  - The Class.forName() method is used to dynamically load the driver class into memory.
  - Loading the driver automatically registers it with the DriverManager.

Syntax

Class.forName("com.mysql.cj.jdbc.Driver");

Or

Class.forName(" com.mysql.cj.jdbc.Driver ").newInstance();

Creating a Connection

- The getConnection() method of the DriverManager class is used to establish a connection between a Java application and a database.

Syntax

Connection con = DriverManager.getConnection(URL, UserName, Password);

- *URL* is an address that points to your database.
  "jdbc:mysql://localhost:3306/testdb"
- *UserName*
  is the user of the database.
- *Password*
  is the password for the database.

Creating a Statement

- The createStatement() method of the Connection interface is used to create a Statement object.
  - A Statement object is responsible for sending SQL queries to the database and retrieving results.

Syntax

Statement stmt = con.createStatement();

- Uses of Statement Objects
  - Query the database
  - Update the database

Creating a Statement

- There are three types of JDBC statements:
- Statement
  - Used for simple SQL statements with no parameters.
    - Example: SELECT * FROM students;
- PreparedStatement - Extends Statement
  - Used for precompiled SQL statements that may include input parameters.
    - Example:

```
PreparedStatement ps = con.prepareStatement( "SELECT * FROM students WHERE id = ?");
ps.setInt(1, 101);
```

- CallableStatement - Extends PreparedStatement
  - Used to execute stored procedures that may contain both input and output parameters.
    - CallableStatement cs = con.prepareCall("{call getStudent(?, ?)}");
    - cs.setInt(1, 101); // input parameter
    - cs.registerOutParameter(2, Types.VARCHAR); // output parameter

Executing Queries

The Statement interface provides three main methods to execute SQL commands:

- **execute(String sql)**
  - Can be used for any SQL statement.
  - Returns true if the result is a ResultSet, otherwise false.

```
boolean result = stmt.execute("SELECT * FROM students");
```

- **executeUpdate(String sql)**
  - Used for data manipulation: INSERT, DELETE, UPDATE, CREATE TABLE, etc.
  - Returns the number of rows affected

```
int rows = stmt.executeUpdate("UPDATE students SET grade = 'A' WHERE id = 101");
```

- **executeQuery(String sql)**
  - Used for SELECT statements only.
  - Returns a ResultSet object containing the query results..

```
ResultSet rs = stmt.executeQuery("SELECT * FROM students");
```

Executing Queries

- The executeQuery() method of the Statement interface is used to execute SQL SELECT queries on the database.
  - It returns a ResultSet object containing all the rows retrieved from the query.

Syntax

ResultSet rs = stmt.executeQuery(query);

Ex:

```
String query = "SELECT * FROM students";
ResultSet rs = stmt.executeQuery(query);
```

Close the Connection

- After completing database operations, it is important to close the connection to free resources.
  - Closing the Connection object automatically closes the Statement and ResultSet objects created from it.
  - Use the close() method of the Connection interface..

Syntax

con.close();

- con is the connection object created at the beginning.

Prepared Statements

- Prepared Statements are used for SQL queries that are executed multiple times with different values.
  - The SQL statement is parsed and compiled by the database only once.
  - Column values can be set later using placeholders.
  - Use ? as a placeholder for values in the query.
  - Helps prevent SQL injection and improves performance.

  - Homework
    - Why use Prepared Statements?

Prepared Statements - Example

Example of parameterized query:

```
String sql = "INSERT INTO emp VALUES (?, ?, ?)";
PreparedStatement ps = con.prepareStatement(sql);

// Set values for placeholders
ps.setInt(1, 101);
ps.setString(2, "John");
ps.setDouble(3, 5000.0);

ps.executeUpdate();
```

ResultSet

- A **ResultSet** object **holds the data returned by executing a Statement or PreparedStatement query**.
- The **rows are retrieved sequentially**.
- A ResultSet maintains a **cursor** pointing to the current row.
- The **next()** method moves the cursor to the **next row**.

```
ResultSet rs = stmt.executeQuery("SELECT * FROM students");
while (rs.next()) {
        int id = rs.getInt("id");
        String name = rs.getString("name");
        System.out.println(id + " : " + name);
}
rs.close(); // Always close the ResultSet
```

ResultSet Methods

- ResultSet provides methods to retrieve column values from the current row.
- Accessing Data by Column Index
  - getType(int columnIndex)
  - Returns the value of the specified column index in the given type.
  - Important: Column indices start at 1, not 0.
    - rs.getString(1); // Get first column as String
    - rs.getInt(2); // Get second column as int
    - rs.getTime(3); // Get third column as Time
- Accessing Data by Column Name
  - getType(String columnName)
  - Returns the value of the specified column using its name.
  - Slightly less efficient than using column index.
    - rs.getString("name");
rs.getInt("id");
rs.getBoolean("isActive");

Handling Errors with Exceptions

- In JDBC, exceptions may occur during database operations.
- To handle errors, use try-catch-finally blocks.
- This ensures the program can recover gracefully and the database remains consistent.
  - For example, you can rollback transactions or close database resources when an exception occurs.

Connect to mysql Database- Example

Following information are needed for the mysql database:

- Driver class
  - The driver class for the mysql database is com.mysql.cj.jdbc.Driver.
- Connection URL
  - The connection URL for the mysql database is jdbc:mysql://localhost:3306/tecruh
    - jdbc is the API
    - mysql is the database
    - localhost is the server's name on which mysql is running, we may also use IP address,
    - 3306 is the port number and tecruh is the database name.
  - Username
    - The default username for the mysql database is root.
  - Password
    - Password is given by the user at the time of installing the mysql database.
    - In this example, we are going to use 1234 as the password.

Connect to mysql Database - Example

- Create “tecruh” database and “student” table.

```
CREATE DATABASE tecruh;

CREATE TABLE student(
        id int(10) PRIMARY KEY,
        name varchar(40),
        age int(3));
```

Connect to mysql Database - Example

```
// 1. importing the mysql library
import java.sql.*;  //importing the mysql library
class MysqlCon{
    public static void main(String args[]){
        try{
            // 2. register the driver
          Class.forName("com.mysql.cj.jdbc.Driver");


            // 3. establish the connection
         Connection con=DriverManager.getConnection("jdbc:mysql://localhost:3306/tecruh","root",“1234");
          //here tecruh is database name, root is username and 1234 is password


            // 4. create statement
          Statement stmt=con.createStatement();
```

Connect to mysql Database - Example

```
        // 5. execute the query to retrieve data from table
        ResultSet rs=stmt.executeQuery("SELECT * FROM student");


        // 6. traverse through the result set
        while(rs.next())
                System.out.println(rs.getInt(1)+""+rs.getString(2)+""+rs. getInt(3));


        // 7. close the connection
        con.close();
      }
      catch(Exception e){
        System.out.println(e.getMessage());
    }
  }
}
```

HandsOn

- Let's connect to MYSQL

- Complete "ICT2132 - OOPP - Practical 20"

Summary

- What is an API
- JDBC Introduction
- Why JDBC
- JDBC Components
- JDBC Architecture
- JDBC Drivers
- Basic steps to use a database in Java
- Handling Exceptions

References

- https://docs.oracle.com/javase/tutorial/jdbc/basics/index.html

- https://dev.mysql.com/doc/relnotes/connector-j/en/

- How To Program (Early Objects)
  - By H .Deitel and P. Deitel
- Headfirst Java
  - By Kathy Sierra and Bert Bates

Questions ???



Thank You